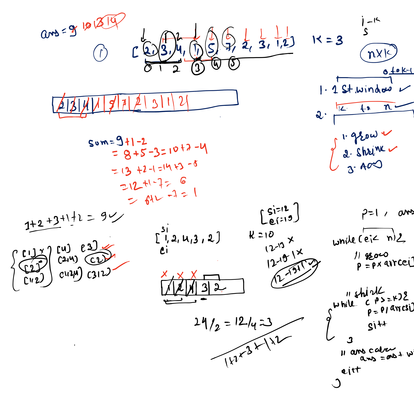
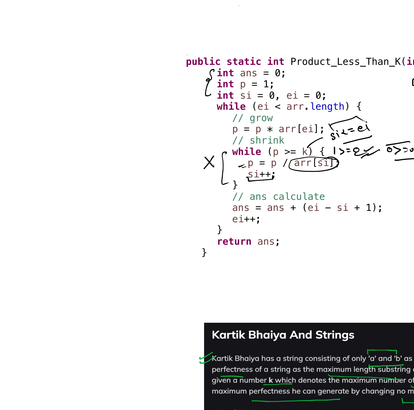







```
public static int Product_Less_Than_K(int[] arr, int k) {
    int ans = 0;
    int p = 1;
    int si = 0, ei = 0;
    while (ei < arr.length) {
      // grow
      p = p * arr[ei];
      // shrink
      while (p >= k) {
        p = p / arr[si];
        si++;
      }
      // ans calculate
      ans = ans + (ei - si + 1);
      ei++;
    }
    return ans;
  }
```

Kartik Bhaiya And Strings

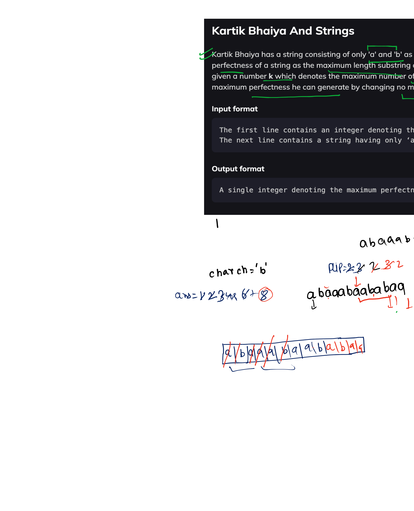
Kartik Bhaiya has a string consisting of only 'a' and 'b' as the characters. Kartik Bhaiya describes perfectness of a string as the maximum length substring of equal characters. Kartik Bhaiya is given a number k which denotes the maximum number of characters he can change. Find the maximum perfectness he can generate by changing no more than k characters.

Input format

The first line contains an integer denoting the value of K.
The next line contains a string having only 'a' and 'b' as the characters.

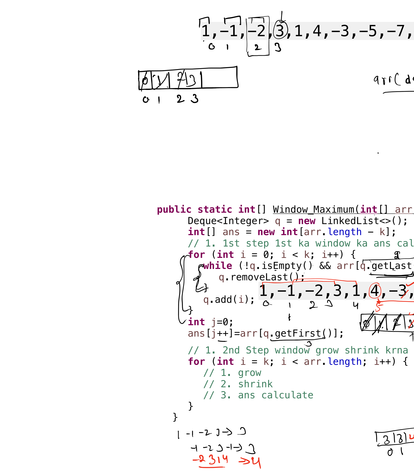
Output format

A single integer denoting the maximum perfectness achievable.

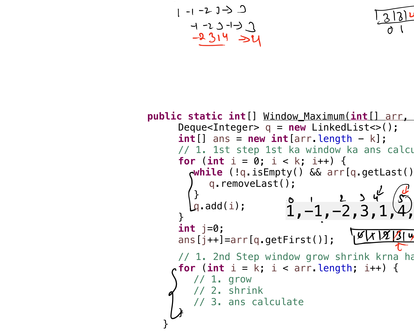


Example 1:

```
Input: nums = [1,3,-1,-3,5,3,6,7], k = 3
Output: [3,3,5,5,6,7]
Explanation:
Window position                Max
---------------               -----
[1  3  -1] -3  5  3  6  7       3
 1 [3  -1  -3] 5  3  6  7       3
 1  3 [-1  -3  5] 3  6  7       5
 1  3  -1 [-3  5  3] 6  7       5
 1  3  -1  -3 [5  3  6] 7       6
 1  3  -1  -3  5 [3  6  7]      7
```




```
public static int[] Window_Maximum(int[] arr, int k) {
    Deque<Integer> q = new LinkedList<>();
    int[] ans = new int[arr.length - k];
    // 1. 1st step 1st ka window ka ans calculate krna hai
    for (int i = 0; i < k; i++) {
      while (!q.isEmpty() && arr[q.getLast()] < arr[i]) {
        q.removeLast();
      }
      q.add(i);
    }
    int j=0;
    ans[j++]=arr[q.getFirst()];
    // 1. 2nd Step window grow shrink krna hai
    for (int i = k; i < arr.length; i++) {
      // 1. grow
      // 2. shrink
      // 3. ans calculate
    }
  }
```




```
public static int[] Window_Maximum(int[] arr, int k) {
    Deque<Integer> q = new LinkedList<>();
    int[] ans = new int[arr.length - k];
    // 1. 1st step 1st ka window ka ans calculate krna hai
    for (int i = 0; i < k; i++) {
      while (!q.isEmpty() && arr[q.getLast()] < arr[i]) {
        q.removeLast();
      }
      q.add(i);
    }
    int j=0;
    ans[j++]=arr[q.getFirst()];
    // 1. 2nd Step window grow shrink krna hai
    for (int i = k; i < arr.length; i++) {
      // 1. grow
      // 2. shrink
      // 3. ans calculate
    }
  }
```